Operating System Lab 1 (PCC-304-AIM) – Question Bank with Answers

Third Year AI & ML, SPPU (2024 Course) · 10 questions per assignment · Oral/Practical preparation

Assignment 1: Linux Environment and File Management (CO1)

Q1. What is Linux and what are its key features?

Linux is an open-source, Unix-like operating system kernel created by Linus Torvalds in 1991. Key features: multi-user, multitasking, portability, security (permissions), hierarchical file system, shell/command-line, and free availability under GPL.

Q2. What is the difference between a kernel and a shell?

The kernel is the core of the OS that manages CPU, memory, devices and system calls. The shell is a command interpreter that takes user commands and passes them to the kernel (e.g., bash, zsh, sh).

Q3. Explain the Linux directory structure.

It is a single hierarchy rooted at `/`. Important directories: `/bin` (essential binaries), `/etc` (configuration), `/home` (user home), `/var` (logs, variable data), `/tmp` (temporary files), `/usr` (user programs), `/dev` (device files), `/proc` (process info), `/root` (root's home).

Q4. List commands for file and directory operations.

`ls` (list), `cd` (change directory), `pwd` (current path), `mkdir` / `rmdir` (create/remove directory), `touch` (create file), `cp` (copy), `mv` (move/rename), `rm` (delete), `cat` / `less` / `head` / `tail` (view), `find` / `locate` (search), `wc` (count).

Q5. Explain Linux file permissions with an example.

Each file has read (r=4), write (w=2) and execute (x=1) permissions for owner, group and others. `-rwxr-xr--` means owner has rwx, group has r-x, others have r--. `chmod 754 file` sets this.

Q6. What is the difference between `chmod` and `chown`?

`chmod` changes the permission bits of a file (e.g., `chmod u+x script.sh`). `chown` changes ownership of the file (e.g., `chown user:group file`). `chgrp` changes only the group.

Q7. What is the use of `ps`, `top` and `kill`?

`ps` shows a snapshot of running processes (`ps -ef`, `ps aux`). `top` shows a live view of processes with CPU and memory use. `kill PID` sends a signal to a process; `kill -9 PID` forcibly terminates it.

Q8. What are hard links and soft (symbolic) links?

A hard link is another name for the same inode; it survives deletion of the original name and cannot cross file systems (`ln file link`). A soft link is a separate file that stores the path of the target; it breaks if the target is deleted and can cross file systems (`ln -s file link`).

Q9. Explain `grep`, `sort`, `cut` and pipes.

`grep` searches for patterns in text, `sort` orders lines, `cut` extracts columns/fields. A pipe `|` sends the output of one command as input to another, e.g., `ps -ef | grep bash | wc -l`.

Q10. Name some basic system administration commands.

`useradd` / `passwd` / `userdel` (user management), `df` (disk space), `du` (directory usage), `free` (memory), `uname -a` (system info), `uptime`, `date`, `who`, `sudo` (run as superuser), `apt` / `yum` (package management), `systemctl` (service management).

Assignment 2: Shell Scripting and System Automation (CO1)

Q1. What is a shell script?

A shell script is a text file containing a sequence of shell commands executed by the shell interpreter. It automates repetitive tasks. It begins with a shebang line such as `#!/bin/bash` and must be made executable with `chmod +x script.sh`.

Q2. How do you declare and use variables in bash?

Assign without spaces: `name="OS"`; use with `$name` or `${name}`. Read input using `read name`. Command output is captured with `$(date)`. Special variables: `$0` (script name), `$1..$9` (arguments), `$#` (argument count), `$?` (exit status), `$$` (PID).

Q3. Explain decision-making statements with syntax.

```
if [ $a -gt $b ]; then
  echo "a is greater"
elif [ $a -eq $b ]; then
  echo "equal"
else
  echo "b is greater"
fi
```

The `case ... esac` statement handles multiple-choice branching.

Q4. List the relational and file test operators.

Numeric: `-eq`, `-ne`, `-gt`, `-lt`, `-ge`, `-le`. String: `=`, `!=`, `-z` (empty), `-n` (non-empty). File: `-f` (regular file), `-d` (directory), `-e` (exists), `-r`, `-w`, `-x` (permissions), `-s` (non-zero size).

Q5. Explain the loops in shell scripting.

`for i in 1 2 3; do ...; done` iterates over a list. `while [ cond ]; do ...; done` runs while the condition is true. `until [ cond ]; do ...; done` runs until the condition becomes true. `break` and `continue` control loop flow.

Q6. How are functions defined and called in bash?

```
greet() {
  echo "Hello $1"
}
greet Student
```

Arguments are accessed by `$1`, `$2`. Use `return` for status codes and `local` for local variables.

Q7. Write a script to find the factorial of a number.

```
#!/bin/bash
read -p "Enter n: " n
f=1
for (( i=1; i<=n; i++ )); do
  f=$((f*i))
done
echo "Factorial = $f"
```

Q8. How do you perform file handling in shell scripts?

Check existence with `[ -f file ]`; redirect output with `>` (overwrite) and `>>` (append); read a file line-by-line with `while read line; do ...; done < file`. Use `cat`, `cp`, `mv`, `rm` for operations.

Q9. What is the difference between `>`, `>>`, `<` and `2>`?

`>` redirects stdout and overwrites, `>>` appends, `<` takes input from a file, `2>` redirects stderr (error stream). `&>` redirects both.

Q10. How can you automate tasks using cron?

`cron` is a job scheduler. Edit with `crontab -e`. Format: `minute hour day month weekday command`. Example: `0 2 * * * /home/user/backup.sh` runs a backup script daily at 2 AM. A typical automation script handles backup, log cleanup or disk-usage alerts.

Assignment 3: Process Creation, Scheduling and Performance Analysis (CO2, CO4, CO5)

Q1. What is a process? How does it differ from a program?

A program is a passive set of instructions stored on disk; a process is a program in execution with its own address space, PID, registers, stack and resources.

Q2. Explain the process states and the Process Control Block (PCB).

States: New, Ready, Running, Waiting/Blocked, Terminated. The PCB stores process ID, state, program counter, CPU registers, scheduling information, memory limits and open files. It is used for context switching.

Q3. Explain the `fork()` system call.

`fork()` creates a child process that is a copy of the parent. It returns 0 to the child, the child's PID to the parent, and -1 on failure. Example: `pid_t p = fork(); if(p==0) printf("child"); else printf("parent");`

Q4. What are `exec()`, `wait()` and `exit()`?

`exec()` family replaces the current process image with a new program. `wait()` makes the parent wait until a child terminates. `exit()` terminates a process and returns a status to the parent.

Q5. What are zombie and orphan processes?

A zombie is a terminated process whose entry remains in the process table because the parent has not called `wait()`. An orphan is a running child whose parent has terminated; it is adopted by `init`/`systemd`.

Q6. Explain FCFS scheduling with its drawbacks.

First-Come First-Served is non-preemptive; processes run in order of arrival. Simple, but suffers from the convoy effect (short processes wait behind long ones) and can give high average waiting time.

Q7. Explain SJF scheduling.

Shortest Job First selects the process with the smallest CPU burst. It gives the minimum average waiting time. It can be non-preemptive or preemptive (SRTF). Drawbacks: starvation of long processes and difficulty predicting burst time.

Q8. Explain Round Robin scheduling and the effect of time quantum.

Each process gets a fixed time quantum in cyclic order; it is preemptive and suited to time-sharing. A very large quantum behaves like FCFS; a very small quantum causes excessive context-switch overhead.

Q9. Define turnaround time, waiting time and response time. Give formulas.

Turnaround Time = Completion Time − Arrival Time. Waiting Time = Turnaround Time − Burst Time. Response Time = First CPU Time − Arrival Time. Also important: CPU utilization and throughput.

Q10. Compare FCFS, SJF and Round Robin.

FCFS: simple, non-preemptive, poor average waiting time. SJF: optimal waiting time, possible starvation. Round Robin: fair, good response time, overhead depends on quantum. Choice depends on workload and system type (batch vs interactive).

Assignment 4: Multithreading and Resource Utilization (CO2, CO3, CO4)

Q1. What is a thread? How is it different from a process?

A thread is the smallest unit of CPU execution within a process. Threads of the same process share code, data and open files but have their own stack, registers and program counter. Threads are lighter and faster to create and switch than processes.

Q2. What are the benefits of multithreading?

Responsiveness, resource sharing, economy (less overhead), scalability on multicore processors, and better CPU utilization.

Q3. What are POSIX threads (pthreads)?

Pthreads is the IEEE POSIX standard API for thread creation and synchronization in C on Unix/Linux. Compile with `gcc prog.c -lpthread`.

Q4. Explain `pthread_create()` and `pthread_join()`.

`pthread_create(&tid, NULL, function, arg)` starts a new thread executing `function`.
`pthread_join(tid, NULL)` makes the calling thread wait for the specified thread to finish.
`pthread_exit()` terminates the calling thread.

Q5. Explain the multithreading models.

Many-to-One (many user threads mapped to one kernel thread), One-to-One (each user thread has a kernel thread, used by Linux/Windows), and Many-to-Many (multiplexes user threads onto several kernel threads).

Q6. Differentiate user-level and kernel-level threads.

User-level threads are managed by a library without kernel knowledge: fast but a blocking call blocks the whole process. Kernel-level threads are managed by the OS: can run in parallel on multiple cores but have more overhead.

Q7. How can you measure CPU utilization and memory usage of a threaded program?

Use `top` / `htop` for live usage, `time ./a.out` for real/user/sys time, `ps -o pid,%cpu,%mem,nlwp -p PID` for threads, and `/proc/PID/status` for memory details. In code, `clock_gettime()` measures execution time.

Q8. How does increasing the number of threads affect performance?

Performance improves until the thread count matches available cores; beyond that, context-switching, cache contention and synchronization overhead can reduce speed. Memory use grows with each thread's stack.

Q9. What is a race condition? How is it avoided in threads?

A race condition occurs when multiple threads access shared data concurrently and the result depends on execution order. It is avoided using mutex locks, semaphores or atomic operations.

Q10. Write the structure of a basic pthread program.

```
#include <pthread.h>
#include <stdio.h>
void* work(void* arg){ printf("Thread %d\n", *(int*)arg); return NULL; }
int main(){
  pthread_t t; int id=1;
  pthread_create(&t, NULL, work, &id);
  pthread_join(t, NULL);
  return 0;
}
```

Assignment 5: Inter-Process Communication and Shared Resource Management (CO2, CO3, CO5)

Q1. What is Inter-Process Communication (IPC)? Why is it needed?

IPC is a set of mechanisms that allow processes to exchange data and synchronize actions. It is needed for information sharing, modularity, computation speed-up and convenience.

Q2. List the IPC mechanisms in Linux.

Pipes, named pipes (FIFO), message queues, shared memory, semaphores, signals and sockets.

Q3. Explain pipes with an example.

An (unnamed) pipe is a unidirectional byte stream between related processes (parent-child), created with `pipe(int fd[2])`: `fd[0]` is the read end and `fd[1]` is the write end. The parent writes using `write(fd[1], ...)` and the child reads with `read(fd[0], ...)`.

Q4. Differentiate between a pipe and a named pipe (FIFO).

A pipe is temporary and works only between related processes. A FIFO has a name in the file system (created by `mkfifo`) and can be used by unrelated processes; it persists until deleted.

Q5. Explain shared memory IPC and its system calls.

Shared memory lets processes map the same memory region, giving the fastest IPC since no kernel copy is needed after setup. System calls: `shmget()` (create/get segment), `shmat()` (attach), `shmdt()` (detach), `shmctl()` (control/remove).

Q6. Explain message queues and their system calls.

A message queue is a kernel-maintained linked list of messages that processes can send and receive with a message type. Calls: `msgget()`, `msgsnd()`, `msgrcv()`, `msgctl()`.

Q7. Why does shared memory need synchronization?

The kernel does not control access to shared memory, so concurrent reads/writes can cause race conditions. Semaphores or mutexes must be used to coordinate access.

Q8. Compare pipes, message queues and shared memory.

Pipes: simple, unidirectional, stream-based. Message queues: structured messages with types, kernel-managed, moderate speed. Shared memory: fastest, needs explicit synchronization, suited to large data.

Q9. How do you analyze communication efficiency and memory utilization in IPC?

Measure data-transfer time for different message sizes using `clock_gettime()`, monitor memory with `ipcs -m`, `ipcs -q`, `free -h` and `/proc`, and compare throughput and latency across mechanisms.

Q10. What are the `ipcs` and `ipcrm` commands?

`ipcs` displays active IPC resources (shared memory, queues, semaphores). `ipcrm` removes an IPC object by ID, e.g., `ipcrm -m shmid`, which helps clean up leftover resources.

Assignment 6: Synchronization Mechanisms for Concurrent Systems (CO2, CO3, CO5)

Q1. What is the critical section problem?

When several processes/threads access shared data, the code segment that accesses it is the critical section. A solution must satisfy mutual exclusion, progress and bounded waiting.

Q2. What is a semaphore? Explain its types.

A semaphore is an integer variable accessed only through two atomic operations: `wait()` (P, decrement) and `signal()` (V, increment). A binary semaphore takes values 0 or 1 (works like a lock); a counting semaphore takes any non-negative value and controls access to a pool of resources.

Q3. What is a mutex lock? How does it differ from a semaphore?

A mutex is a locking mechanism with ownership: only the thread that locked it can unlock it. A semaphore is a signalling mechanism with no ownership and can be used by different processes to signal events. In pthreads: `pthread_mutex_lock()` and `pthread_mutex_unlock()`.

Q4. Which POSIX semaphore functions are used in Linux?

`sem_init()` (initialize), `sem_wait()` (decrement/block), `sem_post()` (increment), `sem_destroy()` (remove). Include `<semaphore.h>`.

Q5. Explain the Producer–Consumer (bounded buffer) problem and its solution.

A producer inserts items into a fixed-size buffer and a consumer removes them. The producer must wait if the buffer is full, the consumer must wait if it is empty, and access must be exclusive. Solution uses three semaphores: `mutex = 1`, `empty = N`, `full = 0`. Producer: `wait(empty); wait(mutex); add item; signal(mutex); signal(full);`. Consumer: `wait(full); wait(mutex); remove item; signal(mutex); signal(empty);`.

Q6. Explain the Readers–Writers problem.

Multiple readers may read shared data simultaneously, but a writer needs exclusive access. In the first readers-writers solution (reader priority), a `read_count` variable with a `mutex` and a `wrt` semaphore is used; the first reader locks `wrt` and the last reader releases it. Writers may starve in this version.

Q7. What is the Dining Philosophers problem?

Five philosophers share five chopsticks; each needs two to eat. Naive solutions can cause deadlock if every philosopher picks up the left chopstick. Remedies: allow at most four philosophers at the table, pick both chopsticks atomically, or use asymmetric ordering.

Q8. What is a race condition? Give an example.

It occurs when the outcome depends on the order of access to shared data. Example: two threads incrementing `counter++` simultaneously can both read the same value and lose an update, because `counter++` is not atomic (load, add, store).

Q9. What are deadlock and starvation in synchronization?

Deadlock: processes wait forever for resources held by each other. Starvation: a process waits indefinitely because others are always given priority. Incorrect use of semaphores (wrong order of `wait`) can cause deadlock.

Q10. How does synchronization affect system performance?

Synchronization guarantees correctness but adds overhead: threads block and wait, context switches increase, and parallelism reduces (lock contention). Performance can be evaluated by measuring execution time, throughput and CPU utilization with and without locks, and by minimizing critical-section size.

Assignment 7: Deadlock Detection, Avoidance and Recovery (CO2, CO3, CO4)

Q1. What is a deadlock?

A situation where a set of processes are blocked, each holding a resource and waiting for a resource held by another process in the set.

Q2. State the four necessary conditions for deadlock.

Mutual exclusion, Hold and wait, No preemption, and Circular wait. All four must hold simultaneously.

Q3. What is a Resource Allocation Graph (RAG)?

A directed graph with process nodes and resource nodes. A request edge goes P→R, an assignment edge goes R→P. A cycle indicates a possible deadlock (a definite deadlock if every resource has a single instance).

Q4. What are the methods for handling deadlock?

Deadlock prevention (negate one of the four conditions), deadlock avoidance (Banker's algorithm), deadlock detection and recovery, and the ostrich approach (ignore it, used by Linux/Windows).

Q5. Explain Banker's Algorithm.

A deadlock avoidance algorithm that grants a request only if the system remains in a safe state. Data structures: `Available`, `Max`, `Allocation`, `Need = Max − Allocation`. The safety algorithm finds a sequence of processes that can all finish using `Work = Available`.

Q6. What is a safe state and a safe sequence?

A state is safe if the system can allocate resources to each process in some order and still avoid deadlock. That order is the safe sequence <P1, P2, ..., Pn>. An unsafe state may lead to deadlock but does not necessarily do so.

Q7. Explain the Resource-Request algorithm of Banker's.

When Pi requests `Request[i]`: (1) if Request ≤ Need, continue, else error; (2) if Request ≤ Available, continue, else wait; (3) pretend to allocate: Available −= Request, Allocation += Request, Need −= Request; (4) run the safety algorithm – if safe, grant; otherwise roll back and wait.

Q8. How is deadlock detected?

For single-instance resources, a wait-for graph is used and a cycle means deadlock. For multiple-instance resources, a detection algorithm similar to Banker's (using `Allocation` and `Request` matrices) finds processes that cannot finish.

Q9. How can the system recover from deadlock?

Process termination (abort all deadlocked processes, or abort one at a time until the cycle breaks) and resource preemption (take resources from some processes, select a victim, rollback, and avoid starvation).

Q10. How can deadlock be prevented? Give examples.

Mutual exclusion: make resources sharable where possible. Hold and wait: request all resources at once. No preemption: allow preemption of resources. Circular wait: impose a total ordering of resource types and request in increasing order.

Assignment 8: Memory Management and Page Replacement Strategies (CO4, CO5)

Q1. What is memory management and why is it needed?

It is the OS function that tracks memory, allocates it to processes, and reclaims it. It provides protection, relocation, sharing and efficient utilization.

Q2. Explain paging.

Paging divides physical memory into fixed-size frames and logical memory into pages of the same size. A page table maps pages to frames. A logical address = (page number, offset). It avoids external fragmentation but may cause internal fragmentation.

Q3. Explain segmentation.

Segmentation divides a program into logical units (code, data, stack) of variable size. A logical address = (segment number, offset), translated using a segment table with base and limit. It matches the user's view but may cause external fragmentation.

Q4. Differentiate between paging and segmentation.

Paging: fixed-size, invisible to user, causes internal fragmentation. Segmentation: variable-size, visible to user, causes external fragmentation.

Q5. What is virtual memory and demand paging?

Virtual memory lets processes run even if only partly in physical memory, giving a large logical address space. In demand paging, pages are loaded only when needed; a reference to a page not in memory causes a page fault.

Q6. What is a page fault? Describe how it is handled.

A page fault occurs when a process accesses a page not in main memory. The OS traps, finds the page on disk, selects a free frame (or a victim page), loads the page, updates the page table, and restarts the instruction.

Q7. Explain the FIFO page replacement algorithm.

The oldest page in memory is replaced first, using a queue. Simple but may replace heavily used pages and suffers from Belady's anomaly (more frames can cause more faults).

Q8. Explain the LRU page replacement algorithm.

Replaces the page that has not been used for the longest time, based on past usage. It performs well and does not suffer from Belady's anomaly, but needs hardware support (counters or a stack) to implement.

Q9. Explain the Optimal page replacement algorithm.

Replaces the page that will not be used for the longest time in the future. It gives the lowest page-fault rate but is impossible to implement in practice because it needs future knowledge; it is used as a benchmark.

Q10. How do you compare page replacement algorithms?

Using the page fault count, hit ratio = hits / total references, and miss ratio = faults / total references. Example: for reference string 7,0,1,2,0,3,0,4,2,3 with 3 frames, Optimal has the fewest faults, then LRU, then FIFO. Effective Access Time = $(1-p)\times$memory access time + $p\times$page-fault service time.

Assignment 9: File System and Disk Scheduling Analysis (CO4, CO5)

Q1. What is a file system? List its functions.

A file system organizes how data is stored and retrieved on storage devices. Functions: file creation/deletion, directory management, allocation of space, access control, and mapping logical files to physical blocks.

Q2. Explain contiguous file allocation.

Each file occupies a set of contiguous blocks. Advantages: fast sequential and direct access, simple. Disadvantages: external fragmentation and difficulty in growing files.

Q3. Explain linked file allocation.

Each file is a linked list of disk blocks, each block pointing to the next. No external fragmentation and files can grow easily, but direct access is slow and pointers consume space (FAT is a variant).

Q4. Explain indexed file allocation.

An index block holds pointers to all the blocks of the file. It supports direct access without external fragmentation, but has index-block overhead. Unix inodes use direct, indirect, double and triple-indirect pointers.

Q5. What is disk scheduling and why is it required?

Disk scheduling decides the order in which pending I/O requests are served. Its goal is to minimize seek time (head movement) and improve throughput and access time.

Q6. Define seek time, rotational latency and transfer time.

Seek time: time to move the head to the required track. Rotational latency: time for the required sector to rotate under the head. Transfer time: time to read/write the data. Disk access time = seek + rotational latency + transfer time.

Q7. Explain FCFS and SSTF disk scheduling.

FCFS serves requests in arrival order; fair but may cause large head movement. SSTF serves the request closest to the current head position; lower average seek time but can cause starvation of far requests.

Q8. Explain SCAN and C-SCAN disk scheduling.

SCAN (elevator algorithm): the head moves in one direction serving requests until the end, then reverses. C-SCAN: the head moves in one direction serving requests, then jumps back to the start without serving on the return, giving more uniform wait times.

Q9. Calculate total head movement: queue 98, 183, 37, 122, 14, 124, 65, 67; head at 53 (FCFS).

Order: 53→98→183→37→122→14→124→65→67. Movements: 45+85+146+85+108+110+59+2 = 640 cylinders. For SSTF on the same queue: 53→65→67→37→14→98→122→124→183 = 12+2+30+23+84+24+2+59 = 236.

Q10. How do you compare disk scheduling algorithms?

Compare total head movement, average seek time, throughput and fairness. SSTF and SCAN usually give lower seek time than FCFS; C-SCAN gives more uniform waiting; SCAN/C-SCAN do better under heavy load. Storage utilization depends on the allocation method used (fragmentation).

Assignment 10: Containerization and Resource Monitoring for AI/ML Workloads (CO1, CO2, CO5)

Q1. What is containerization?

Containerization packages an application with its libraries, dependencies and configuration into a lightweight, isolated unit called a container that runs consistently on any system sharing the host OS kernel.

Q2. What is Docker? Explain its main components.

Docker is a platform for building, shipping and running containers. Components: Docker Engine/Daemon, Docker Client (CLI), Docker Image (read-only template), Docker Container (running instance), Dockerfile (build instructions) and Docker Hub (registry).

Q3. Differentiate between a virtual machine and a container.

VMs include a full guest OS on a hypervisor: heavy, slow to start, strong isolation. Containers share the host kernel: lightweight, start in seconds, use less memory and storage, with weaker isolation than VMs.

Q4. Which Linux kernel features make containers possible?

Namespaces provide isolation (PID, network, mount, user, UTS, IPC). Cgroups (control groups) limit and account for CPU, memory and I/O. Union file systems (e.g., OverlayFS) provide layered images.

Q5. What is a Dockerfile? Write a simple one for an ML application.

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

It defines the base image, working directory, dependencies, source code and startup command.

Q6. List the common Docker commands.

`docker build -t myapp .` (build image), `docker run -d -p 5000:5000 myapp` (run container), `docker ps` (running containers), `docker images`, `docker stop <id>`, `docker rm <id>`, `docker logs <id>`, `docker exec -it <id> bash`.

Q7. How do you monitor container resource usage?

`docker stats` shows live CPU, memory, network and block I/O per container. Use `docker top <id>` for processes, `docker system df` for disk usage, and `docker inspect` for configuration.

Q8. Which Linux tools monitor CPU, memory, storage and processes?

CPU/processes: `top`, `htop`, `ps`, `mpstat`. Memory: `free -h`, `vmstat`. Storage: `df -h`, `du -sh`, `iostat`. Others: `/proc`, `sar`, `iotop`, `nmon`.

Q9. How do you limit resources for a container?

Use run flags: `docker run --memory="512m" --cpus="1.5" myapp`. These use cgroups to restrict memory and CPU so one workload cannot starve others.

Q10. What are the benefits of container-based deployment for AI/ML workloads?

Reproducible environments (same libraries/versions everywhere), portability across laptop, server and cloud, quick deployment and scaling, isolation of dependencies, efficient resource usage compared with VMs, and easy integration with CI/CD and orchestration tools like Kubernetes. GPU workloads can be run using the NVIDIA Container Toolkit.